

md5_algorithm.c

```
#include "md5_algorithm.h"
```

```
unsigned leftRotate(unsigned x, int n) {  
    return (x << n) | (x >> (32 - n));  
}
```

```
void md5(unsigned int *message, unsigned int *result) {
```

```
    uint32_t A, B, C, D, i, F, g, Dtemp;
```

```
    const uint32_t a0 = 0x67452301;
```

```
    const uint32_t b0 = 0xefcdab89;
```

```
    const uint32_t c0 = 0x98badcfe;
```

```
    const uint32_t d0 = 0x10325476;
```

```
    A = a0;
```

```
    B = b0;
```

```
    C = c0;
```

```
    D = d0;
```

```
    for (i = 0; i < 64; i++) {
```

```
        if (i < 16) {
```

```
            F = (B & C) | (~B & D);
```

```
            g = i;
```

```
        } else if (i < 32) {
```

```
            F = (D & B) | (~D & C);
```

```
            g = (5 * i + 1) % 16;
```

```
        } else if (i < 48) {
```

```
F = B ^ C ^ D;  
g = (3 * i + 5) % 16;  
} else {  
    F = C ^ (B | ~D);  
    g = (7 * i) % 16;  
}
```

```
Dtemp = D;  
D = C;  
C = B;  
B += leftRotate((A + F + K[i] + message[g]), s[i]);  
A = Dtemp;  
}
```

```
result[0] = a0 + A;  
result[1] = b0 + B;  
result[2] = c0 + C;  
result[3] = d0 + D;  
}
```

md5_algorithm.h

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
const uint32_t s[64] = {  
    7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22,  
    5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20,  
    4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23,  
    6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21  
};
```

```
const uint32_t K[64] = {  
    0xd76aa478, 0xe8c7b756, 0x242070db, 0xc1bdcee5,  
    0xf57c0faf, 0x4787c62a, 0xa8304613, 0xfd469501,  
    0x698098d8, 0x8b44f7af, 0xffff5bb1, 0x895cd7be,  
    0x6b901122, 0xfd987193, 0xa679438e, 0x49b40821,  
    0xf61e2562, 0xc040b340, 0x265e5a51, 0xe9b6c7aa,  
    0xd62f105d, 0x02441453, 0xd8a1e681, 0xe7d3fbc8,  
    0x21e1cde6, 0xc33707d6, 0xf4d50d87, 0x455a14ed,  
    0xa9e3e905, 0xfcefa3f8, 0x676f02d9, 0x8d2a4c8a,  
    0xffffa394, 0x8771f681, 0x6d9d6122, 0xfde5380c,  
    0xa4beea44, 0x4bdecfa9, 0xf6bb4b60, 0xbebfbc70,  
    0x289b7ec6, 0xeaa127fa, 0xd4ef3085, 0x04881d05,  
    0xd9d4d039, 0xe6db99e5, 0x1fa27cf8, 0xc4ac5665,  
    0xf4292244, 0x432aff97, 0xab9423a7, 0xfc93a039,
```

```
0x655b59c3, 0x8f0ccc92, 0xffeff47d, 0x85845dd1,  
0x6fa87e4f, 0xfe2ce6e0, 0xa3014314, 0x4e0811a1,  
0xf7537e82, 0xbd3af235, 0x2ad7d2bb, 0xeb86d391  
};
```

```
unsigned leftRotate(unsigned x, int n);  
void md5(unsigned int *message, unsigned int *result);
```

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <time.h>
#include <sys/mman.h>
#include "hwlib.h"
#include "socal/socal.h"
#include "socal/hps.h"
#include "socal/alt_gpio.h"
#include "hps_0.h"
#include <assert.h>
#include <pthread.h>

// Define a macro for printing messages
#define PRINT_ENABLED 0 // Set to 1 to enable printing, 0 to disable
#define print(fmt, ...) if (PRINT_ENABLED) printf(fmt, ##__VA_ARGS__)
#define CURRENT_TIME time(NULL)
#define NUMBER_OF_ENGINES 16
#define LW_SIZE 0x00200000
#define LWHPS2FPGA_BASE 0xff200000

const static size_t timeout = 10;
volatile uint32_t *md5ControlBase = NULL;
volatile uint32_t *md5DataBase = NULL;
int success, total;
```

```
const uint32_t testSeq[] =  
{0x01680208,  
 0x13ab80bb,  
 0xcb8b2c30,  
 0xb9657582,  
 0xa3793c48,  
 0x103f26be,  
 0x0b78dac4,  
 0x5c433348,  
 0x4de99287,  
 0xeff0be7c,  
 0x00808533,0x00000000,  
 0x00000000,  
 0x00000000,  
 0x00000150,  
 0x00000000};
```

```
typedef struct {  
    size_t engine;  
    uint32_t message;  
} ThreadArgs;
```

```
typedef struct {  
    struct md5Control {  
        volatile uint32_t* md5_start;  
        volatile uint32_t* md5_reset;
```

```

    volatile uint32_t* md5_wr;
    volatile uint32_t* md5_done;
} md5Control;

struct md5Data {
    volatile uint32_t* writedata;
    volatile uint32_t* writeaddr;
    volatile uint32_t* readaddr;
    volatile uint32_t* readdata;
} md5Data;
} s_md5Registers;

s_md5Registers md5Registers = {
    .md5Control = {0},
    .md5Data = {0}
};

void setRegister(volatile uint32_t* registerName, volatile uint32_t word, char*
registerNameString)
{
    alt_write_word(registerName, word);

    print("Asserting: \t\tRegister(0x%08x, %s) = 0x%08x\n", registerName, registerNameString,
alt_read_word(registerName));
}

void clearRegister(volatile uint32_t* registerName, char* registerNameString){
    alt_write_word(registerName, 0x0);

    print("De-Asserting: \t\tRegister(0x%08x, %s) = 0x%08x\n", registerName,
registerNameString, alt_read_word(registerName));
}

```

```
}
```

```
void checkRegister(volatile uint32_t* registerName, uint32_t word, char* registerNameString)
```

```
{
```

```
    time_t startTime = time(NULL);
```

```
    print("Reading: \t\tRegister(0x%08x, %s) = 0x%08x\n", registerName, registerNameString,  
alt_read_word(registerName));
```

```
    while (1)
```

```
    {
```

```
        if (alt_read_word(registerName) == word)
```

```
            return ;
```

```
        assert(diffTime(CURRENT_TIME, startTime) < timeout);
```

```
    }
```

```
}
```

```
void md5Reset(){
```

```
    print("===\t Resetting \t===\n");
```

```
    setRegister(md5Registers.md5Control.md5_reset, 0x1, "md5_reset");
```

```
    clearRegister(md5Registers.md5Control.md5_wr, "md5_wr");
```

```
    clearRegister(md5Registers.md5Control.md5_start, "md5_start");
```

```
    clearRegister(md5Registers.md5Data.writeaddr, "writeaddr");
```

```
    clearRegister(md5Registers.md5Data.readaddr, "readaddr");
```

```
    clearRegister(md5Registers.md5Control.md5_reset, "md5_reset");
```

```
}
```

```
void md5Start(){
```

```
    print("===\t Starting Engine \t===\n");
```



```

setRegister(md5Registers.md5Control.md5_start, 0x1, "md5_start");
clearRegister(md5Registers.md5Control.md5_start, "md5_start");
checkRegister(md5Registers.md5Control.md5_done, 0xFFFFFFFF, "md5_done");
}

```

```

void md5PrintDigest(){
    print("===\t Reading Digest \t===\n");
    uint32_t totalDigest[] = {0, 0, 0, 0};
    int i;
    for (i = 0; i < 4; i++)
    {
        setRegister(md5Registers.md5Data.readaddr, i, "readaddr");
        totalDigest[i] = alt_read_word(md5Registers.md5Data.readdata);
    }
    printf("Digest: ");
    for (i = 0; i < 4; i++)
        printf("%08x", totalDigest[3 - i]);
}

```

```

void md5HardwareAlgorithm(){
    // Reset
    md5Reset();
    // Set data
    print("===\t Setting Data \t===\n");
    setRegister(md5Registers.md5Control.md5_wr, 0x1, "md5_wr");
    int i;
    for (i = 0; i < 16; i++)

```

```

{
    print("Stage: %d\n", i);
    setRegister(md5Registers.md5Data.writeaddr, i, "writeaddr");
    setRegister(md5Registers.md5Data.writedata, testSeq[i], "writedata");
}

clearRegister(md5Registers.md5Control.md5_wr, "md5_wr");

// Start engine
md5Start();

// Read digest
md5PrintDigest();
}

```

```

void writeEngine(void* args){
    ThreadArgs* parameters = (ThreadArgs*)args;
    size_t engine = parameters->engine;
    uint32_t message = parameters->message;
    setRegister(md5Registers.md5Data.writeaddr, engine, "writeaddr");
    setRegister(md5Registers.md5Data.writedata, message, "writedata");
}

```

```

int main(int argc, char **argv){
    int fd;
    success = 0; total = 0;

    //map address space of fpga for software to access here
    if((fd = open("/dev/mem", ( O_RDWR | O_SYNC ) ) ) == -1 ) {
        printf( "ERROR: could not open \"/dev/mem\"...\n" );
    }
}

```

```

    return( 1 );
}

void* virtual_base = mmap( NULL, LW_SIZE, ( PROT_READ | PROT_WRITE ), MAP_SHARED, fd,
LWHP2FPGA_BASE);

if( virtual_base == MAP_FAILED ) {
    printf( "ERROR: mmap() failed...\n" );
    close( fd );
    return(1);
}

//initialize the addresses
md5ControlBase = virtual_base + ((uint32_t)( MD5_CONTROL_0_BASE) );
md5DataBase = virtual_base + ((uint32_t)(MD5_DATA_0_BASE));

uint32_t *md5Control = &md5Registers.md5Control.md5_start;
uint32_t *md5Data = &md5Registers.md5Data.writedata;
size_t i;

for (i = 0; i < 4; i++)
{
    printf("ControlBase offset = 0x%08x\n", md5ControlBase + i);
    md5Control[i] = md5ControlBase + i;
}

for (i = 0; i < 4; i++)
{

```

```
    printf("ControlBase offset = 0x%08x\n", md5DataBase + i);  
    md5Data[i] = md5DataBase + i;  
}
```

```
printf("----->md5 HW in serial<-----\n");  
md5HardwareAlgorithm();  
printf( "\n===== \n" );  
printf( "Compare software digest\n" );
```

```
uint32_t message[] = {  
    0x01680208, 0x13ab80bb, 0xcb8b2c30, 0xb9657582,  
    0xa3793c48, 0x103f26be, 0x0b78dac4, 0x5c433348,  
    0x4de99287, 0xe7f0be7c, 0x00808533, 0x00000000,  
    0x00000000, 0x00000000, 0x00000150, 0x00000000  
};
```

```
size_t len = sizeof(message) / sizeof(message[0]);  
uint32_t digest[4];  
md5(message, digest);  
printf("MD5 Hash: %08x%08x%08x%08x\n", digest[3], digest[2], digest[1], digest[0]);  
printf("\n");  
printf( "\n===== \n" );
```

```
printf("----->md5 HW in parallel<-----\n");  
md5Reset();  
setRegister(md5Registers.md5Control.md5_wr, 0x1, "md5_wr");
```

```

pthread_t threads[NUMBER_OF_ENGINES];
ThreadArgs args[NUMBER_OF_ENGINES];

for (i = 0; i < NUMBER_OF_ENGINES; i++)
{
    args[i].engine = i;
    args[i].message = testSeq[i];
    pthread_create(&threads[i], NULL, (void *)writeEngine, &args[i]);
}

for (i = 0; i < NUMBER_OF_ENGINES; ++i)
    pthread_join(threads[i], NULL);

clearRegister(md5Registers.md5Control.md5_wr, "md5_wr");
md5Start();
md5PrintDigest();

printf( "\n===== \n" );
printf( "Compare software digest\n" );
printf("MD5 Hash: %08x%08x%08x%08x\n", digest[3], digest[2], digest[1], digest[0]);
printf("\n");

// clean up our memory mapping and exit
if( munmap( virtual_base, LW_SIZE) != 0 ) {
    print( "ERROR: munmap() failed...\n" );
    close( fd );
    return( 1 );
}

```

```
}
```

```
close( fd );
```

```
return 0;
```

```
}
```

md5.vhd

library IEEE;

use IEEE.std_logic_1164.all;

use IEEE.numeric_std.all;

ENTITY md5 IS

PORT(

CLOCK_50, HPS_DDR3_RZQ, HPS_ENET_RX_CLK, HPS_ENET_RX_DV : IN STD_LOGIC;

HPS_DDR3_ADDR : OUT STD_LOGIC_VECTOR(14 DOWNTO 0);

HPS_DDR3_BA : OUT STD_LOGIC_VECTOR(2 DOWNTO 0);

HPS_DDR3_CS_N : OUT STD_LOGIC;

HPS_DDR3_CK_P, HPS_DDR3_CK_N, HPS_DDR3_CKE : OUT STD_LOGIC;

HPS_USB_DIR, HPS_USB_NXT, HPS_USB_CLKOUT : IN STD_LOGIC;

HPS_ENET_RX_DATA : IN STD_LOGIC_VECTOR(3 DOWNTO 0);

HPS_SD_DATA, HPS_DDR3_DQS_N : INOUT STD_LOGIC_VECTOR(3 DOWNTO 0);

HPS_DDR3_DQS_P : INOUT STD_LOGIC_VECTOR(3 DOWNTO 0);

HPS_ENET_MDIO : INOUT STD_LOGIC;

HPS_USB_DATA : INOUT STD_LOGIC_VECTOR(7 DOWNTO 0);

HPS_DDR3_DQ : INOUT STD_LOGIC_VECTOR(31 DOWNTO 0);

HPS_SD_CMD : INOUT STD_LOGIC;

HPS_ENET_TX_DATA, HPS_DDR3_DM : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);

HPS_DDR3_ODT, HPS_DDR3_RAS_N, HPS_DDR3_RESET_N : OUT STD_LOGIC;

HPS_DDR3_CAS_N, HPS_DDR3_WE_N : OUT STD_LOGIC;

HPS_ENET_MDC, HPS_ENET_TX_EN : OUT STD_LOGIC;

HPS_USB_STP, HPS_SD_CLK, HPS_ENET_GTX_CLK : OUT STD_LOGIC

);

END ENTITY md5;

ARCHITECTURE Behaviour OF md5 IS

--instantiate the soc_system component here

component soc_system is

```
port (  
    clk_clk : in std_logic := 'X'; -- clk  
    hps_0_h2f_reset_reset_n : out std_logic; -- reset_n  
    hps_io_hps_io_emac1_inst_TX_CLK : out std_logic; -- hps_io_emac1_inst_TX_CLK  
    hps_io_hps_io_emac1_inst_TXD0 : out std_logic; -- hps_io_emac1_inst_TXD0  
    hps_io_hps_io_emac1_inst_TXD1 : out std_logic; -- hps_io_emac1_inst_TXD1  
    hps_io_hps_io_emac1_inst_TXD2 : out std_logic; -- hps_io_emac1_inst_TXD2  
    hps_io_hps_io_emac1_inst_TXD3 : out std_logic; -- hps_io_emac1_inst_TXD3  
    hps_io_hps_io_emac1_inst_RXD0 : in std_logic := 'X'; -- hps_io_emac1_inst_RXD0  
    hps_io_hps_io_emac1_inst_MDIO : inout std_logic := 'X'; -- hps_io_emac1_inst_MDIO  
    hps_io_hps_io_emac1_inst_MDC : out std_logic; -- hps_io_emac1_inst_MDC  
    hps_io_hps_io_emac1_inst_RX_CTL : in std_logic := 'X'; -- hps_io_emac1_inst_RX_CTL  
    hps_io_hps_io_emac1_inst_TX_CTL : out std_logic; -- hps_io_emac1_inst_TX_CTL  
    hps_io_hps_io_emac1_inst_RX_CLK : in std_logic := 'X'; -- hps_io_emac1_inst_RX_CLK  
    hps_io_hps_io_emac1_inst_RXD1 : in std_logic := 'X'; -- hps_io_emac1_inst_RXD1  
    hps_io_hps_io_emac1_inst_RXD2 : in std_logic := 'X'; -- hps_io_emac1_inst_RXD2  
    hps_io_hps_io_emac1_inst_RXD3 : in std_logic := 'X'; -- hps_io_emac1_inst_RXD3  
    hps_io_hps_io_sdio_inst_CMD : inout std_logic := 'X'; -- hps_io_sdio_inst_CMD  
    hps_io_hps_io_sdio_inst_D0 : inout std_logic := 'X'; -- hps_io_sdio_inst_D0  
    hps_io_hps_io_sdio_inst_D1 : inout std_logic := 'X'; -- hps_io_sdio_inst_D1  
    hps_io_hps_io_sdio_inst_CLK : out std_logic; -- hps_io_sdio_inst_CLK  
    hps_io_hps_io_sdio_inst_D2 : inout std_logic := 'X'; -- hps_io_sdio_inst_D2  
    hps_io_hps_io_sdio_inst_D3 : inout std_logic := 'X'; -- hps_io_sdio_inst_D3
```



```

hps_io_hps_io_usb1_inst_D0 : inout std_logic := 'X'; -- hps_io_usb1_inst_D0
hps_io_hps_io_usb1_inst_D1 : inout std_logic := 'X'; -- hps_io_usb1_inst_D1
hps_io_hps_io_usb1_inst_D2 : inout std_logic := 'X'; -- hps_io_usb1_inst_D2
hps_io_hps_io_usb1_inst_D3 : inout std_logic := 'X'; -- hps_io_usb1_inst_D3
hps_io_hps_io_usb1_inst_D4 : inout std_logic := 'X'; -- hps_io_usb1_inst_D4
hps_io_hps_io_usb1_inst_D5 : inout std_logic := 'X'; -- hps_io_usb1_inst_D5
hps_io_hps_io_usb1_inst_D6 : inout std_logic := 'X'; -- hps_io_usb1_inst_D6
hps_io_hps_io_usb1_inst_D7 : inout std_logic := 'X'; -- hps_io_usb1_inst_D7
hps_io_hps_io_usb1_inst_CLK : in std_logic := 'X'; -- hps_io_usb1_inst_CLK
hps_io_hps_io_usb1_inst_STP : out std_logic; -- hps_io_usb1_inst_STP
hps_io_hps_io_usb1_inst_DIR : in std_logic := 'X'; -- hps_io_usb1_inst_DIR
hps_io_hps_io_usb1_inst_NXT : in std_logic := 'X'; -- hps_io_usb1_inst_NXT
memory_mem_a : out std_logic_vector(14 downto 0); -- mem_a
memory_mem_ba : out std_logic_vector(2 downto 0); -- mem_ba
memory_mem_ck : out std_logic; -- mem_ck
memory_mem_ck_n : out std_logic; -- mem_ck_n
memory_mem_cke : out std_logic; -- mem_cke
memory_mem_cs_n : out std_logic; -- mem_cs_n
memory_mem_ras_n : out std_logic; -- mem_ras_n
memory_mem_cas_n : out std_logic; -- mem_cas_n
memory_mem_we_n : out std_logic; -- mem_we_n
memory_mem_reset_n : out std_logic; -- mem_reset_n
memory_mem_dq : inout std_logic_vector(31 downto 0) := (others => 'X'); -- mem_dq
memory_mem_dqs : inout std_logic_vector(3 downto 0) := (others => 'X'); -- mem_dqs
memory_mem_dqs_n : inout std_logic_vector(3 downto 0) := (others => 'X'); --
mem_dqs_n
memory_mem_odt : out std_logic; -- mem_odt

```

```

memory_mem_dm : out std_logic_vector(3 downto 0); -- mem_dm
memory_oct_rzqin : in std_logic := 'X'; -- oct_rzqin
reset_reset_n : in std_logic := 'X'; -- reset_n

md5_data_0_md5_data_m_writedata : out std_logic_vector(31 downto 0); --
m_writedata

md5_data_0_md5_data_m_writeaddr : out std_logic_vector(31 downto 0); --
m_writeaddr

md5_data_0_md5_data_m_readaddr : out std_logic_vector(31 downto 0); --
m_readaddr

md5_data_0_md5_data_m_readdata : in std_logic_vector(31 downto 0) := (others =>
'X'); -- m_readdata

md5_control_0_md5_control_m_start : out std_logic_vector(31 downto 0); -- m_start
md5_control_0_md5_control_m_reset : out std_logic_vector(31 downto 0); -- m_reset
md5_control_0_md5_control_m_wr : out std_logic; -- m_wr

md5_control_0_md5_control_m_done : in std_logic_vector(31 downto 0) := (others =>
'X') -- m_done

);
end component soc_system;

```

COMPONENT md5_group

```

PORT(
    clk, wr : IN STD_LOGIC;
    reset, start : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
    writedata : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
    writeaddr : IN STD_LOGIC_VECTOR(8 DOWNTO 0);
    readaddr : IN STD_LOGIC_VECTOR(6 DOWNTO 0);
    done : OUT STD_LOGIC_VECTOR(31 DOWNTO 0);
    readdata : OUT STD_LOGIC_VECTOR(31 DOWNTO 0)
);

```

END COMPONENT;

--SIGNALS instantiated here

SIGNAL reset_reset_n, md5_wr : STD_LOGIC;

SIGNAL md5_input_reset, md5_done : STD_LOGIC_VECTOR(31 DOWNTO 0);

SIGNAL md5_readdata, md5_input_start : STD_LOGIC_VECTOR(31 DOWNTO 0);

SIGNAL md5_writedata, md5_writeaddr, md5_readaddr : STD_LOGIC_VECTOR(31 DOWNTO 0);

BEGIN

--port map soc_system here

u0 : component soc_system

port map (

clk_clk => CLOCK_50,

reset_reset_n => '1',

memory_mem_a => HPS_DDR3_ADDR,

memory_mem_ba => HPS_DDR3_BA,

memory_mem_ck => HPS_DDR3_CK_P,

memory_mem_ck_n => HPS_DDR3_CK_N,

memory_mem_cke => HPS_DDR3_CKE,

memory_mem_cs_n => HPS_DDR3_CS_N,

memory_mem_ras_n => HPS_DDR3_RAS_N,

memory_mem_cas_n => HPS_DDR3_CAS_N,

memory_mem_we_n => HPS_DDR3_WE_N,

memory_mem_reset_n => HPS_DDR3_RESET_N,

memory_mem_dq => HPS_DDR3_DQ,

memory_mem_dqs => HPS_DDR3_DQS_P,

memory_mem_dqs_n => HPS_DDR3_DQS_N,
memory_mem_odt => HPS_DDR3_ODT,
memory_mem_dm => HPS_DDR3_DM,
memory_oct_rzqin => HPS_DDR3_RZQ,
hps_io_hps_io_emac1_inst_TX_CLK => HPS_ENET_GTX_CLK,
hps_io_hps_io_emac1_inst_TXD0 => HPS_ENET_TX_DATA(0),
hps_io_hps_io_emac1_inst_TXD1 => HPS_ENET_TX_DATA(1),
hps_io_hps_io_emac1_inst_TXD2 => HPS_ENET_TX_DATA(2),
hps_io_hps_io_emac1_inst_TXD3 => HPS_ENET_TX_DATA(3),
hps_io_hps_io_emac1_inst_RXD0 => HPS_ENET_RX_DATA(0),
hps_io_hps_io_emac1_inst_MDIO => HPS_ENET_MDIO,
hps_io_hps_io_emac1_inst_MDC => HPS_ENET_MDC,
hps_io_hps_io_emac1_inst_RX_CTL => HPS_ENET_RX_DV,
hps_io_hps_io_emac1_inst_TX_CTL => HPS_ENET_TX_EN,
hps_io_hps_io_emac1_inst_RX_CLK => HPS_ENET_RX_CLK,
hps_io_hps_io_emac1_inst_RXD1 => HPS_ENET_RX_DATA(1),
hps_io_hps_io_emac1_inst_RXD2 => HPS_ENET_RX_DATA(2),
hps_io_hps_io_emac1_inst_RXD3 => HPS_ENET_RX_DATA(3),
hps_io_hps_io_sdio_inst_CMD => HPS_SD_CMD,
hps_io_hps_io_sdio_inst_D0 => HPS_SD_DATA(0),
hps_io_hps_io_sdio_inst_D1 => HPS_SD_DATA(1),
hps_io_hps_io_sdio_inst_CLK => HPS_SD_CLK,
hps_io_hps_io_sdio_inst_D2 => HPS_SD_DATA(2),
hps_io_hps_io_sdio_inst_D3 => HPS_SD_DATA(3),
hps_io_hps_io_usb1_inst_D0 => HPS_USB_DATA(0),
hps_io_hps_io_usb1_inst_D1 => HPS_USB_DATA(1),
hps_io_hps_io_usb1_inst_D2 => HPS_USB_DATA(2),

```

hps_io_hps_io_usb1_inst_D3 => HPS_USB_DATA(3),
hps_io_hps_io_usb1_inst_D4 => HPS_USB_DATA(4),
hps_io_hps_io_usb1_inst_D5 => HPS_USB_DATA(5),
hps_io_hps_io_usb1_inst_D6 => HPS_USB_DATA(6),
hps_io_hps_io_usb1_inst_D7 => HPS_USB_DATA(7),
hps_io_hps_io_usb1_inst_CLK => HPS_USB_CLKOUT,
hps_io_hps_io_usb1_inst_STP => HPS_USB_STP,
hps_io_hps_io_usb1_inst_DIR => HPS_USB_DIR,
hps_io_hps_io_usb1_inst_NXT => HPS_USB_NXT,
hps_0_h2f_reset_reset_n => reset_reset_n,
md5_data_0_md5_data_m_writedata => md5_writedata, --
md5_data_0_md5_data.m_writedata
md5_data_0_md5_data_m_writeaddr => md5_writeaddr, -- .m_writeaddr
md5_data_0_md5_data_m_readaddr => md5_readaddr, -- .m_readaddr
md5_data_0_md5_data_m_readdata => md5_readdata, -- .m_readdata
md5_control_0_md5_control_m_start => md5_input_start, --
md5_control_0_md5_control.m_start
md5_control_0_md5_control_m_reset => md5_input_reset, -- .m_reset
md5_control_0_md5_control_m_wr => md5_wr, -- .m_wr
md5_control_0_md5_control_m_done => md5_done -- .m_done
);

```

m0 : md5_group

PORT MAP(

```

clk => CLOCK_50,
wr => md5_wr,
reset => md5_input_reset,
start => md5_input_start,

```

```
writedata => md5_writedata,  
writeaddr => md5_writeaddr(8 DOWNT0 0),  
readaddr => md5_readaddr(6 DOWNT0 0),  
done => md5_done,  
readdata => md5_readdata  
);
```

End Behaviour;

md5_control.vhd

```
library IEEE;
```

```
use IEEE.std_logic_1164.all;
```

```
use IEEE.numeric_std.all;
```

```
entity md5_control is
```

```
    port (
```

```
        avs_s0_address : in std_logic_vector(3 downto 0) := (others => '0'); -- avalon slave  
interface
```

```
    avs_s0_write   : in std_logic           := '0';
```

```
    avs_s0_writedata : in std_logic_vector(31 downto 0) := (others => '0');
```

```
    avs_s0_read     : in std_logic           := '0';
```

```
    avs_s0_readdata : out std_logic_vector(31 downto 0);
```

```
    clk            : in std_logic           := '0';      -- clock
```

```
    reset          : in std_logic           := '0';      -- reset
```

```
    md5_start      : out std_logic_vector(31 downto 0);   -- control signals to MD5 core
```

```
    md5_reset      : out std_logic_vector(31 downto 0);
```

```
    md5_wr         : out std_logic;                    -- write enable
```

```
    md5_done       : in std_logic_vector(31 downto 0) := (others => '0') -- status from MD5  
core
```

```
);
```

```
end entity md5_control;
```

```
architecture rtl of md5_control is
```

```
    SIGNAL start_reg, reset_reg : STD_LOGIC_VECTOR(31 DOWNTO 0);
```

```
    SIGNAL wr_reg : STD_LOGIC;
```

```
begin
```

```
    PROCESS(clk, reset)
```

BEGIN

IF(reset = '1') THEN

start_reg <= (OTHERS => '0');

reset_reg <= (OTHERS => '0');

wr_reg <= '0';

avs_s0_readdata <= (OTHERS => '0');

ELSIF(rising_edge(clk)) THEN

wr_reg <= '0'; -- default value

IF(avs_s0_write = '1') THEN

CASE avs_s0_address IS

WHEN "0000" => -- Start register

start_reg <= avs_s0_writedata;

WHEN "0001" => -- Reset register

reset_reg <= avs_s0_writedata;

WHEN "0010" => -- Write control

wr_reg <= avs_s0_writedata(0);

WHEN OTHERS =>

-- Do nothing

END CASE;

ELSIF(avs_s0_read = '1') THEN

CASE avs_s0_address IS

WHEN "0000" => -- Start status

avs_s0_readdata <= start_reg;

WHEN "0001" => -- Reset status

avs_s0_readdata <= reset_reg;

WHEN "0010" => -- Done status


```
        avs_s0_readdata <= md5_done;

    WHEN OTHERS =>

        avs_s0_readdata <= (OTHERS => '0');

    END CASE;

END IF;

END IF;

END PROCESS;


md5_start <= start_reg;
md5_reset <= reset_reg;
md5_wr <= wr_reg;


end architecture rtl;
```

md5_data.vhd

```
library IEEE;
```

```
use IEEE.std_logic_1164.all;
```

```
use IEEE.numeric_std.all;
```

```
entity md5_data is
```

```
    port (
```

```
        avs_s0_address : in std_logic_vector(3 downto 0) := (others => '0'); -- avalon slave  
interface
```

```
    avs_s0_read  : in std_logic          := '0';
```

```
    avs_s0_write : in std_logic          := '0';
```

```
    avs_s0_readdata : out std_logic_vector(31 downto 0);
```

```
    avs_s0_writedata : in std_logic_vector(31 downto 0) := (others => '0');
```

```
    clk          : in std_logic          := '0';      -- clock
```

```
    reset        : in std_logic          := '0';      -- reset
```

```
    md5_writedata : out std_logic_vector(31 downto 0);      -- data to MD5 core
```

```
    md5_writeaddr : out std_logic_vector(31 downto 0);      -- write address
```

```
    md5_readaddr  : out std_logic_vector(31 downto 0);      -- read address
```

```
    md5_readdata  : in std_logic_vector(31 downto 0) := (others => '0') -- data from MD5  
core
```

```
);
```

```
end entity md5_data;
```

```
architecture rtl of md5_data is
```

```
    SIGNAL writedata_reg : STD_LOGIC_VECTOR(31 DOWNTO 0);
```

```
    SIGNAL writeaddr_reg : STD_LOGIC_VECTOR(31 DOWNTO 0);
```

```
    SIGNAL readaddr_reg : STD_LOGIC_VECTOR(31 DOWNTO 0);
```

```
begin
```

```

PROCESS(clk, reset)

BEGIN

    IF(reset = '1') THEN

        writedata_reg <= (OTHERS => '0');
        writeaddr_reg <= (OTHERS => '0');
        readaddr_reg <= (OTHERS => '0');
        avs_s0_readdata <= (OTHERS => '0');

    ELSIF(rising_edge(clk)) THEN

        IF(avs_s0_read = '1') THEN

            CASE avs_s0_address IS

                WHEN "0000" => -- Read data from MD5 core

                    avs_s0_readdata <= md5_readdata;

                WHEN "0001" => -- Current write data

                    avs_s0_readdata <= writedata_reg;

                WHEN "0010" => -- Current write address

                    avs_s0_readdata <= writeaddr_reg;

                WHEN "0011" => -- Current read address

                    avs_s0_readdata <= readaddr_reg;

                WHEN OTHERS =>

                    avs_s0_readdata <= (OTHERS => '0');

            END CASE;

        ELSIF(avs_s0_write = '1') THEN

            CASE avs_s0_address IS

                WHEN "0000" => -- Write data

                    writedata_reg <= avs_s0_writedata;

                WHEN "0001" => -- Write address

                    writeaddr_reg <= avs_s0_writedata;

```

```
        WHEN "0010" => -- Read address
            readaddr_reg <= avs_s0_writedata;

        WHEN OTHERS =>
            -- Do nothing
        END CASE;

    END IF;

END IF;

END PROCESS;


md5_writedata <= writedata_reg;
md5_writeaddr <= writeaddr_reg;
md5_readaddr <= readaddr_reg;


end architecture rtl;
```